Algorithm Analysis for Efficient Job Matching

In Support of UN Sustainable Development Goal 8: Decent Work and Economic Growth

Presented by: Santos · Floranda · Lagarde · Masaya · Catindig · Garay

Executive Summary

100% Implementation

Successful development of both Greedy Ranking and Stable Matching systems using a clean, modular Python architecture.

Empirical Validation

Comparative hardware benchmarks conducted across synthetic datasets ranging from $N = 100$ to $N = 2500$

System Architecture

main.py

The central benchmarking framework responsible for dataset ingestion, hardware metric tracking, and algorithm coordination.

team_alpha.py

Contains the optimized Greedy logic and Sorting paradigms focused on high-speed ranking for massive datasets.

team_beta.py

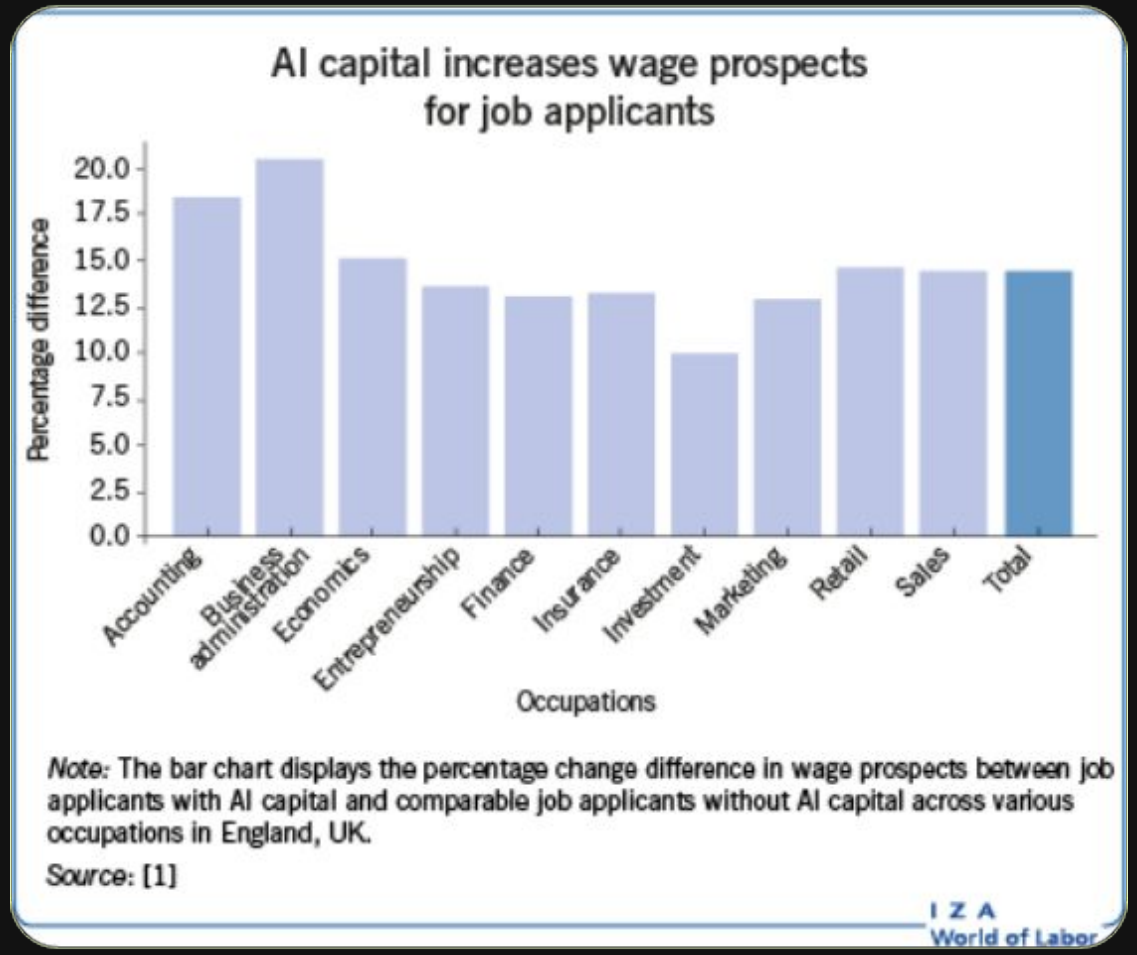
Encapsulates the Gale-Shapley logic with O(1) dictionary lookups to ensure fairness and match stability.

SDG 8 & Community Impact

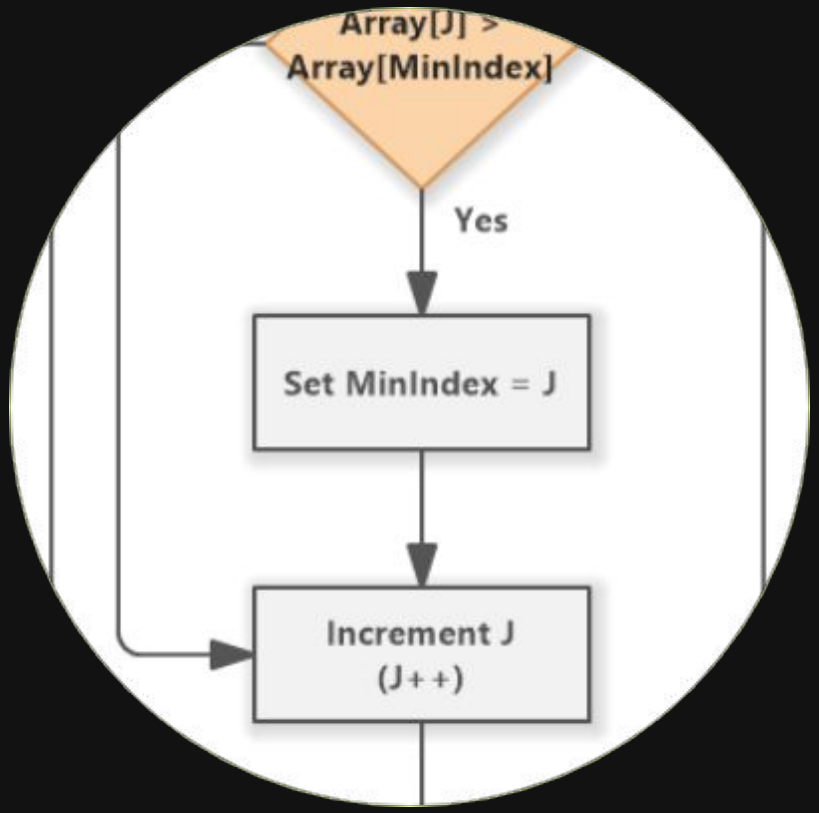
Structural Imbalance

With 5.11M underemployed in the Philippines, traditional keyword-based recruitment is insufficient. Our system targets:

- → SDG 8.5: Productive Employment
- → Reduction of Information Asymmetry
- → Optimized Resource Allocation



Optimized Greedy Logic

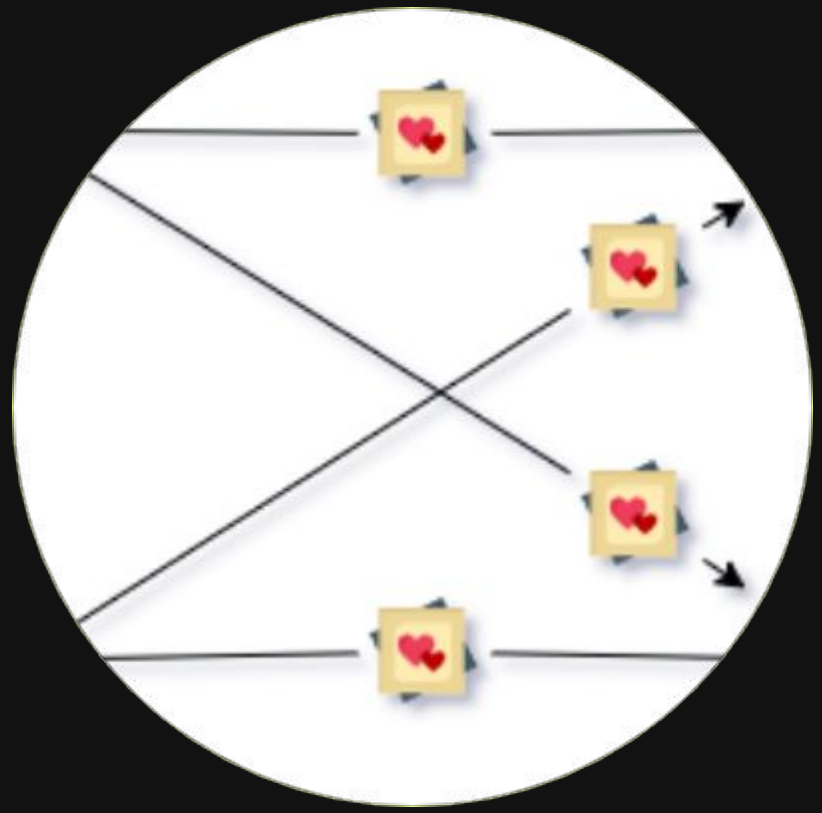


O(n log n) Paradigm

By implementing a composite "Hireability Score" (Skill + Experience), Team Alpha prioritizes candidates with minimal computational overhead.

Ideal for high-volume initial screening where processing millions of applications per minute is required.

Stable Matching Logic



Gale-Shapley Optimization

Utilizing deque structures for O(1) proposer removal, Team Beta ensures that every match is mutually beneficial.

The algorithm guarantees 100% stability, ensuring no applicant and employer would prefer a different pairing.

The Stability Audit

0

Blocking Pairs Found

Mathematical Verification

- Following the matching process, we executed an automated "Blocking Pair" detector across 2,500 assignments.

- The result confirms that the matching is stable, supporting the SDG 8 objective of "fair" and optimized work placement.

Hardware Performance Monitoring

Empirical Metrics

We tracked performance using specialized Python utilities to ensure industrial-grade accuracy:

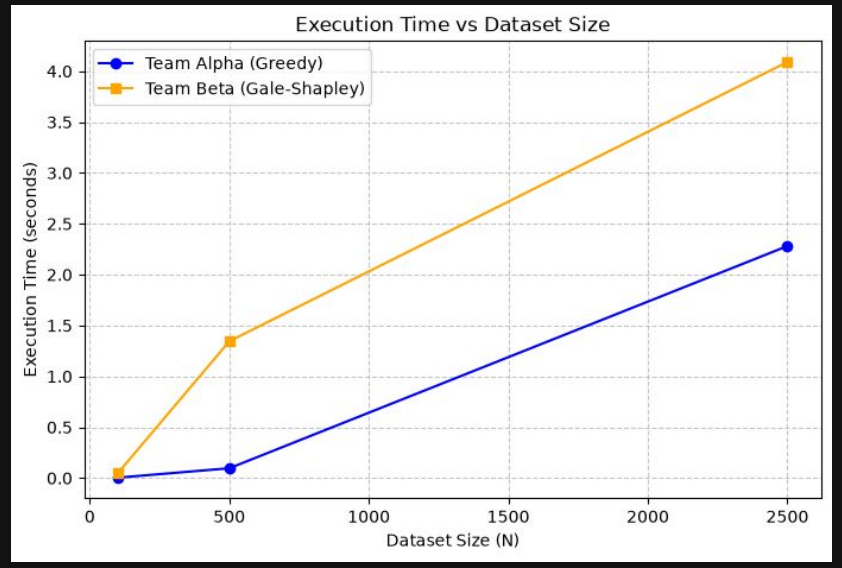
- tracemalloc: Peak RAM Tracking
- time: Execution Duration
- scaling: Tests at 100, 500, and 2500

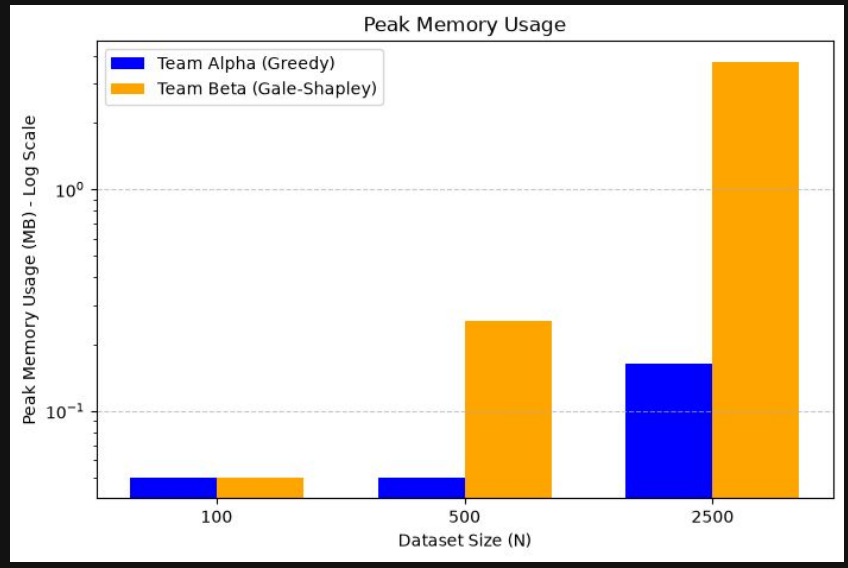
```
C:\Users\Ryan\Documents\CSS133-Job-Matching-SDG8>py main.py
Loading datasets for N=2500...
Data loaded successfully! Let's get matching.

========================================
Starting Team Alpha's Greedy Algorithm...
Greedy Matching complete!
Execution Time: 17.5777 seconds
Peak Memory Usage: 1.17 MB
Total Matches Made: 1228
========================================

========================================
Starting Team Beta's Gale-Shapley Algorithm...
Verifying match stability...
Verification Success: Perfect stability achieved! No blocking pairs exist.
Gale-Shapley Matching complete!
Execution Time: 63.5006 seconds
Peak Memory Usage: 412.95 MB
Total Matches Made: 2500
========================================
```

Comparative Results Data

Dataset Size (N)	Algorithm	Execution Time (s)	Peak Memory (MB)	Matches Made
100	Alpha (Greedy)	0.14 s	0.19 MB	–
100	Beta (Stable)	0.10 s	0.56 MB	–
500	Alpha (Greedy)	1.78 s	0.23 MB	–
500	Beta (Stable)	2.02 s	14.83 MB	–
2500	Alpha (Greedy)	17.74 s	1.12 MB	49%
2500	Beta (Stable)	79.08 s	412.94 MB	100%

Benchmark Visualization





The Gale-Shapley implementation shows significant growth in execution duration as complexity increases to $O(n^2)$.

Limitations & Future Work

- **Limitation Mitigation:** We used our Sorting algorithm as a strict pre-processing step to limit the candidate pool, preventing the Gale-Shapley complexity from crashing the system on massive datasets.
- **Future Work 1:** Compare our hybrid algorithmic approach against modern Machine Learning (ML) recommendation engines.
- **Future Work 2:** Add multi-objective complexity, such as balancing a candidate's skill compatibility with their geographical commute distance.

The Scalability Trade-off

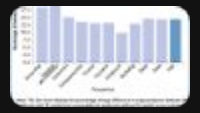
Alpha Insight

Team Alpha is 75% faster at scaling. However, its greedy paradigm matched only 49% of candidates, sacrificing match volume for speed.

Beta Insight

Team Beta ensures 100% Match Rate. However, memory consumption increases exponentially, peaking at 412MB for just 2500 users.

Image Sources



https://wol.iza.org/uploads/articles/694/images/Drydakis-GA-verbessert.png

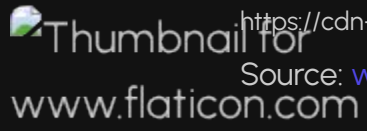
Source: wol.iza.org

https://www.softwareideas.net/i/DirectImage/1759/Selection-Sort--Flowchart-

Source: www.softwareideas.net

https://miro.medium.com/1*KdRrgAcynLdpMTC68ia3Qw.png

Source: medium.com



https://cdn-icons-png.flaticon.com/512/8901/8901603.png

Source: www.flaticon.com

Thank you!